

Heterogeneous Multi-Agent Task-Assignment with Uncertain Execution Times and Preferences

Qinshuang Wei, Vaibhav Srivastava, Vijay Gupta

Abstract—While task assignment problems for a single agent have been widely studied, multi-agent task assignment problems with heterogeneous task preferences or characteristics remain relatively less well-characterized. We study a multi-agent task assignment problem where tasks need to be assigned sequentially to multiple members of a team. The members have heterogeneous capabilities in terms of time taken to complete any task and preferences in terms of reward collected upon completion of a task. We assume that both the reward and execution times for each member to complete any task follow unknown stochastic distributions. The goal of the assigner is to maximize the total reward received by the team over the planning horizon. We propose and analyze a bandit algorithm for this problem. Since the bandit algorithm relies on an optimal task assignment problem being solved repeatedly, we analyze the achievable regret in two cases. If the optimal task assignment can be exactly solved, we show that the regret for our algorithm is the same as that in the single-agent bandit task assignment algorithm. If, on the other hand, the exact solution is not accessible, we compute the regret with respect to an oracle-based algorithm that uses a sub-optimal solver.

I. INTRODUCTION

A multi-agent task assignment problem aims to maximize the reward from task completion by mapping tasks to agents, who may have heterogeneous capabilities. The main challenge in this problem is computational efficiency as the number of tasks and agents increase, particularly if agents have diverse preferences in terms of reward they generate for completing a particular task or characteristics in terms of time taken to complete a task. Recent advancements in theorems applicable to autonomous heterogeneous task assignment problems, such as multi-armed bandit for recurrent tasks [1], [2], crowdsourcing systems for real-world problem-solving requiring human workers [3], [4], and multi-robot coordination [5] have enabled task assignments to heterogeneous individuals with various objectives besides maximizing utilities, such as participants satisfaction [6] **VG: This sentence is not clear. I will suggest separating the works that have considered / posed the problem from works and the works that have proposed solution approaches. Within the latter, separate the works with combinatorial or other algorithms when parameters are known v/s bandit or other approaches when they are not known.**

Due to their importance, multi-agent task assignment problems with heterogeneous preferences when the preferences

or task completion times are known have been studied in the literature. However, when these parameters are unknown a priori, only a few works have considered the problem of *learning* the optimal solution over time. **VG: I think we need a better rest of the paragraph here. We need some organization of the various works (e.g. [7] appears in two different streams. We also need a sentence each for the various streams about why what we want to study is not solved by them.** Some methods consider uncertain outcomes obtained for task completion [1], [2], [7], [8]. By formulating the outcome as a weighted score for both the task completion and performer satisfaction rate, these investigations can adapt to heterogeneous task-assignment processes. Some methods further consider task completion time uncertainty [2], having the potential to reflect personal capabilities for different tasks. Moreover, existing literature also considers the distributed multi-user task assignment that takes user preference into account [7]. Some crowdsourcing systems with heterogeneous workers also estimate workers' abilities or preferences under a bandit setup [3], [4], [9], [10], [11], [12], [13], [14], [15]. To study unknown worker reliabilities at different tasks, [3], [10] use bandit-based task assignment to adapt to different tasks and the task-dependent reliability of workers, [4] uses bi-objective optimization with combinatorial multi-armed bandits, [9], [11] considers stochastic outcomes of assignments representing worker preferences, [12] explores budget-limited task assignment problem with heterogeneous workers in both costs and in the quality of the work, [14], [15] assign tasks to the workers who are more interested in the tasks with higher priorities. Besides, [16] presents a bandit algorithm to select the motor task that yields the best performance for an asynchronous brain-controlled button. [5] studies a scalable task assignment for heterogeneous multi-robot teams, but did not consider the uncertainties in task outcomes or completion times.

We study a task assignment problem in a heterogeneous multi-agent setting, where the members have different task completion times and rewards for task completion for the same task. Tasks arrive sequentially and are assigned to a suitable agent by a central controller subject to individual capability constraints with the goal of maximizing the total team reward over a given planning horizon. The reward and execution time for each member to complete each task follows an unknown distribution. This problem is inspired by the single-player bandit task assignment problem in [2], with major distinctions: First, we generalize it to a multi-player scenario. Second, while the algorithm in [2] is not scalable in certain types of scenarios, we extend it to an algorithm

Q. Wei, and V. Gupta are with the Elmore Family School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47907, USA (emails: wei473@purdue.edu, gupta869@purdue.edu). V. Srivastava is with the Electrical and Computer Engineering, Michigan State University, East Lansing, MI 48824 (email: vaibhav@egr.msu.edu). The work was supported by xxx under Grant xxx.

that suits large-scale heterogeneous bandit task assignment problems. Third, we establish the performance analysis for the extended algorithm to show its effectiveness.

Our main contributions are as follows. First, we formulate the heterogeneous bandit task assignment problem with multi-teammate setting. Second, we propose an efficient task assignment algorithm designed for large-scale heterogeneous bandit task allocation, where obtaining an exact optimal task assignment at each round is computationally intractable. Third, we analyze the bounds for the regret theoretically. When the exact task assignment solution at each round is accessible, the regret for the algorithm is the same as that in the single-agent bandit task assignment algorithm. When the exact solution is not accessible, we analyze the upper bound of a sub-optimal regret for the proposed algorithm. Both the optimal and the sub-optimal regrets have the upper bound $O\left(\frac{1}{C_l} \sqrt{C_u N M \bar{L} T \ln T} + \frac{C_u^2}{C_l^2} N M \bar{L} \ln T\right)$, where C_l, C_u are the minimal and maximal task completion time, respectively, T is the time horizon, N is the number of tasks, M is the number of team members, and $\bar{L}$ is the largest number of tasks that the whole team can execute simultaneously. Last, we validate the proposed algorithm through a case study with 3 teams of different sizes, demonstrating its effectiveness in practical applications.

A. Notations

We let M_{ij} be the ij 'th entry of the matrix M . For all vectors or matrices a, b of the same size, $a \leq b$ means that each entry of a is less or equal to the corresponding entry of b . Given two n by m matrices A, B , we let $C = A \odot B$ represent the dot product of two matrices, i.e., $C_{ij} = A_{ij} B_{ij}$. Further, we define the function $\Sigma(A)$ that sums up all entries in the matrix A . We let $\mathbf{1}_N, \mathbf{0}_N$ represent the ones vector and zeros vector with length N , and let $\mathbf{I}_N$ represent the $N \times N$ identity matrix.

II. PROBLEM FORMULATION

We consider that a team consisting of several different members will execute a set of tasks under resource constraints, e.g., energy, and after completion of each task, will receive a reward. A central planner will assign a set of tasks to the team members at each round to maximize the total team reward during the planning horizon.

We denote the set of all tasks as $\mathcal{V} = (v_1, v_2, \dots, v_N)$, where N is the total number of tasks. We further denote the set of all members in the team as $\mathcal{M} = \{m_1, m_2, \dots, m_M\}$, where M is the total number of members. We let the index set $I_V = \{1, 2, \dots, N\}$, $I_M = \{1, 2, \dots, M\}$, and $I_A = \{(i, m) \mid m \in I_M, i \in I_V\}$. Further, the member m has L_m available source at each round, and will need to spend f_{im} amount of resource per unit of time to execute task v_i for all $m \in I_M$ and $i \in I_V$. We denote the maximal number of tasks the team can execute simultaneously as $\bar{L} = \max\{N, \sum_{m=1}^M L_m\}$.

We denote a *task assignment* for the team as a binary matrix $a \in \{0, 1\}^{N \times M}$ and define a feasible task assignment as below.

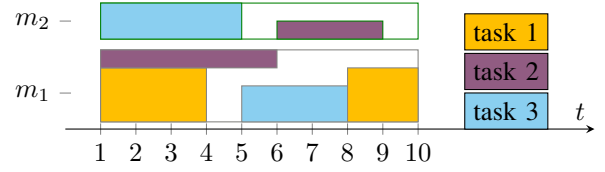


Fig. 1: A feasible task assignment in Example 1. The orange, purple, and blue filled rectangles represent task 1, 2, and 3, respectively. The width of the filled rectangle shows the task completion time, and the height shows how many resources the member needs to spend on the corresponding task. While the grey and green unfilled rectangles represent the total resources that members 1 and 2 have, respectively.

Definition 1 (Feasible Task Assignment). A feasible task assignment a for the team is a task assignment that

- 1) for all $m \in I_M$, the member M_m can execute the set of tasks $\mathcal{V}_m = \{i \in I_V \mid a_{im} = 1\}$ at the same time, that is, $\sum_i f_{im} a_{im} \leq L_m$ based on the resource constraint.
- 2) at most one agent can execute each task simultaneously, that is, $\sum_m a_{im} \leq 1$.

We then denote the family of all feasible task assignments to the team as $\mathcal{A} \subseteq \{0, 1\}^{N \times M}$. Similar to [2], we assume that $\mathcal{A}$ is closed under inclusion, that is, if $a \in \mathcal{A}$ and $b \leq a$, then $b \in \mathcal{A}$.

In each round t , the planner will choose a feasible task assignment $a_t \in \mathcal{A}$ for the team. We let $a_{t,im}$ be the im 'th entry of the binary matrix a_t . For any task assignment with $a_{t,im} = 1$, the member m will start to execute the task i at round t . After completing the task, m will receive a reward $r_{t,im}$ and record the corresponding processing time $c_{t,im}$, i.e., member m will complete task i at the beginning of round $(t + c_{t,im})$ and receive a reward $r_{t,im}$. We assume that $c_{t,im} \in \{C_l, C_l + 1, \dots, C_u\}$, where $C_l, C_u \in \mathbb{N}$ and $1 \leq C_l \leq C_u$, and $r_{t,im} \in [0, 1]$. Further, we assume that for all $m \in I_M$, $\{r_{t,im}, c_{t,im}\}_{i \in I_V}$ follows an unknown distribution over $[0, 1]^N \times \{C_l, C_l + 1, \dots, C_u\}^N$, which is independent of rounds $t = 1, \dots, T \in \mathbb{N}$. However, $c_{t,im}$ and $r_{t,im}$ may be dependent. We denote the matrix $c_t \in \{C_l, C_l + 1, \dots, C_u\}^{N \times M}$ as the completion-time matrix, with the im 'th entry defined as $c_{t,im}$; we denote the matrix $r_t \in [0, 1]^{N \times M}$ as the reward matrix, with the im 'th entry defined as $r_{t,im}$.

Moreover, we restrict the selected task assignment at round t , a_t , to exclude all ongoing tasks $b_t \in \mathcal{A}$, that is, $a_t \in \mathcal{A}_t$, where we define the ongoing tasks b_t and the set of available task assignments $\mathcal{A}_t$ at round t as below:

$$b_{t,im} = \sum_{s=1}^{t-1} a_{s,im} \mathbb{1}[s + c_{s,im} > t], \quad (1)$$

$$\mathcal{A}_t = \{a \in \mathcal{A} \mid a + b_t \in \mathcal{A}\},$$

with $b_{t,im}$ representing the im 'th entry of the binary matrix $b_t \in \{0, 1\}^{N \times M}$.

Example 1. In Fig. 1 we show a feasible task assignment for a team with two members $\mathcal{M} = \{m_1, m_2\}$ and 3 tasks $\mathcal{V} = \{v_1, v_2, v_3\}$, where $L_1 = 2, L_2 = 1, f_{11} = 1.5, f_{21} = f_{22} = 0.5, f_{31} = f_{12} = 1$, and $f_{32} = 0.8$. The completion time are $c_{1,11} = 3, c_{1,21} = 5, c_{5,31} = 3, c_{8,11} = 2, c_{1,32} = 4, c_{6,22} = 3$.

Goal: The goal of the team is to maximize the accumulated reward of the whole team earned by round T , $\sum_{t=1}^T \Sigma(a_t \odot r_t)$.

We define the optimal accumulated reward collected during the planning horizon as

$$OPT = \max_{\pi^* \in \Pi} \mathbb{E} \left[\sum_{t=1}^T \Sigma(r_t \odot \pi_t^*(h_t^*)) \right], \quad (2)$$

where we define a proper policy π as a function that chooses feasible task assignment a_t that satisfies (1) based on history information obtained prior to round t , but not on that after round t , and Π is the set of all proper policy. We let the history h_t at round t consist of all prior task assignments $\{a_s\}_{s=1}^{t-1}$, and all observed rewards and completion times by the beginning of round t , $(r_{s,im}, c_{s,im})$ for all $(i, m) \in I_A$ such that $s + c_{s,im} \leq t$. We denote $\pi = \{\pi_t\}_{t=1,2,\dots}$ such that π_t maps from the history h_t to a feasible assignment $a_t \in \mathcal{A}$.

We then define the α -regret R_T^α —for any $\alpha = \{0\} \cup [1, +\infty)$ —of an algorithm as the difference between the expected accumulated rewards obtained with the α optimal proper policy and the expected accumulated rewards obtained by the proposed algorithm:

$$R_T^\alpha = \frac{1}{1+\alpha} OPT - \mathbb{E} \left[\sum_{t=1}^T \Sigma(r_t \odot a_t) \right], \quad (3)$$

This definition is the same as the $\frac{1}{1+\alpha}$ -regret defined in blocking bandits [1] and $(\frac{1}{1+\alpha}, 1)$ -regret defined in combinatorial bandits [17]. We denote R_T^0 as the *optimal regret* and R_T^α as a *sub-optimal regret* for $\alpha \geq 1$.

This heterogeneous bandit task assignment problem generalizes the stochastic combinatorial semi-bandits [18] and the multi-armed bandit problem with budget constraints [19].

We use q_{im} to denote the per-unit reward function for agent m to complete task i , where $q_{im} = \frac{\bar{r}_{im}}{\bar{c}_{im}}$. By generalizing [2, Proposition 2.2] to a multi-player scenario, we have Proposition 1 below¹.

Proposition 1. *Given any proper policy $\pi = \{\pi_t\}_{t=1,2,\dots} \in \Pi$ and its generated histories $\{h_t\}_{t=1,2,\dots}$, we have $\mathbb{E}[\sum_{t=1}^T \sum_{m,i} r_{t,im} \pi_{im}(h_t)] \leq (T + C_u) \max_{a \in \mathcal{A}} \sum_{m,i} q_{im} a_{im}$.*

Given Proposition 1, we therefore bound the regrets in (4):

$$R_T \leq \frac{1}{\alpha + 1} (T + C_u) \max_{a \in \mathcal{A}} \Sigma(q \odot a) - \mathbb{E} \left[\sum_{t=1}^T \Sigma(r_t \odot a_t) \right]$$

¹Due to page limitation, detailed proofs are contained in the extended version

$$\leq \mathbb{E} \left[\sum_{t=1}^T \Sigma \left(\frac{1}{\alpha + 1} q \odot a^* - r_t \odot a_t \right) \right] + \frac{C_u \bar{L}}{(\alpha + 1) C_l}. \quad (4)$$

III. METHODOLOGY

To assign a proper set of tasks to each member at the beginning of each round, we adopt an algorithm that suits both the exact linear optimization solver and any approximation algorithm in the loop. We utilize a phased-update method proposed in [2] that update the policy at the beginning of each phase, and keeps the same task assignment during the rest of the phase. For further details and analysis on this phased-update method, we refer the interested readers to [2]. However, instead of solving the exact linear optimization problem over $\mathcal{A}$ for every update as in [2], we further revise this algorithm to the more generalized scenario when solving the linear optimization over all feasible task assignments can be hard, and thus relaxation or approximation is necessary.

Algorithm 1 demonstrates the entire process of the algorithm. We divide the time horizon into phases. The algorithm starts with an initialization phase. After initialization, for each phase, we select a task assignment at the beginning of the phase and the members will continue executing the assigned tasks cyclically.

In phase 0, we initialize the algorithm by letting each member m complete each task i that satisfies $f_{im} \leq L_m$ B times as in line 1, where we let $B = \left\lceil 90 \frac{C_u}{C_l} \ln T \right\rceil$ and we assume that $T > MNB C_u$. As [2] points out, the total regret in phase 0 cannot exceed $\mathcal{O}(\frac{C_u MNB}{C_l}) = \mathcal{O}(\frac{C_u^2}{C_l^2} MN \ln T)$, which is trivial when analyzing regret upper bound over the time horizon.

In s 'th phase, where $s = 1, 2, \dots$, as depicted in line 2–13, we first compute the phase length l_s , the UCB estimators $\hat{q}_{im}(t_s)$ through (5), where $t_s = \sum_{k=1}^{s-1} l_k$ is the round after initialization that phase s begins, $\hat{r}_{t,im}$ and $\hat{c}_{t,im}$ are the empirical means of rewards and costs, $V_{t,im}^c$ are the empirical variance of costs, and $T_{im}(t)$ are the times that agent m has completed task i before time t .

$$l_s = C_l \min_{(i,m) \in I_A} T_{im}(t_s) + 2C_u \quad (5a)$$

$$\hat{q}_{im}(t) = \frac{\min\{1, \hat{r}_{im}(t) + d_{im}^r(t)\}}{\max\{C_l, \hat{c}_{im}(t) + d_{im}^c(t)\}} \quad (5b)$$

$$\text{where } d_{im}^r(t) = \sqrt{\frac{1.5 \ln t}{T_{im}(t)}}$$

$$d_{im}^c(t) = \sqrt{\frac{3V_{im}^c(t) \ln t}{T_{im}(t)}} + \frac{9(C_u - C_l) \ln t}{T_{im}(t)}$$

As in line 4, by solving the linear optimization problem (6) below using some given some Algorithm A, we then compute—at the beginning of t_s round—the task assignment a'_s that the team will follow during phase s after completing all tasks that start in phase $s - 1$.

$$\max_{a \in \mathcal{A}} \Sigma(\hat{q}(t_s) \odot a). \quad (6)$$

Then, at each round $t = t_s, t_s + 1, \dots, t_{s+1}$, we first compute the ongoing tasks b_t at time t using (1), and then assign the rest of available tasks a_t in a'_s as in 8, where $a_t \in \mathcal{A}_t$ based on construction. During phase s , member m will repeat each task i such that $a_{s,im} = 1$ right after its completion.

Remark 1 (Algorithm A). *Algorithm A on line 4 of Algorithm 1 can be either an exact optimization or an approximating algorithm. Finding an exact optimal solution to (6)—where we restrict the set of feasible task assignments as in Definition 1—is equivalent to a generalized assignment problem (GAP), and needs to find an optimal solution of a mixed-integer optimization problem (MILP), which is usually NP-hard and is computationally inefficient. In many cases, we are only able to solve the problem (6) approximately through some algorithm such as [20] that only guarantee to find a $(1 + \alpha)$ -approximate solution a' , i.e., $\Sigma(\hat{q}(t_s) \odot a') \geq \frac{1}{1+\alpha} \Sigma(\hat{q}(t_s) \odot a^*)$, where a^* is an optimal solution to (6).*

We call Algorithm 1 an *exact algorithm* when Algorithm A on line 4 provides an exact solution to (6), while we call Algorithm 1 an $1 + \alpha$ *approximate algorithm* or *approximate algorithm* when Algorithm A provides only a $(1 + \alpha)$ -approximate solution to (6).

Algorithm 1 Bandit Algorithm for Task Assigning

Input: C_l, C_u : lower and upper bounds for the processing time. B : length of initialization phase.

- 1: Initialization: For all $i \in I_V$ and $m \in I_M$ such that $f_{im} \leq L_m$, let member m execute the task i for B times. Record the completing round as t_1 , and set $T_{im}(t_1) = B$.
 - 2: **for** $s = 1, 2, \dots$ **do**
 - 3: Set $\hat{q}_{im}(t_s)$ using (5b).
 - 4: Compute $a'_s \in \max_{a \in \mathcal{A}} \Sigma(\hat{q}(t_s) \odot a)$ through Algorithm A.
 - 5: Set $l_s = C_l \min_{(i,m) \in I_A} T_{im}(t_s) + 2C_u$, $t_{s+1} = t_s + l_s$.
 - 6: **for** $t = t_s, t_s + 1, \dots, t_{s+1} - 1$ **do**
 - 7: Compute b_t using (1).
 - 8: If $b_t \leq a'_s$, then $a_t \leftarrow a'_s - b_t$; else $a_t \leftarrow \{0\}^{N \times M}$.
 - 9: **for** $i = 1, \dots, N$ **do**
 - 10: If member m completes the task i at round t and we observe the corresponding reward completion time $r_{t',im}, c_{t',im}$, where t' is the task starting round, update $\hat{r}_{im}(t), \hat{c}_{im}(t), V_{im}^c(t)$. Set $T_{im}(t+1) = T_{im}(t) + 1$.
 - 11: **end for**
 - 12: **end for**
 - 13: **end for**
-

IV. THEORETICAL RESULTS

We explore the performance of the exact and approximating algorithm we proposed in 1. We first derive the optimal regret upper and lower bound obtained from [2] with slight modification in multi-player setup, followed by a special case

of when the exact algorithm is computationally efficient. This special case is useful especially when the size of the problem is large. We will then explore the sub-optimal regret upper bound for the scenarios when it is time-consuming or even unrealistic to obtain the exact solution to linear optimization problem (6), and thus an approximated solution is necessary.

Before the regret analysis, we first define task completion times in initialization and sub-optimal gaps. Following [2], in the later analysis of the paper, we set the initialization task completion times as $B = \lceil 90 \frac{C_u}{C_l} \ln T \rceil$, and we have the properties (7)–(8) for all $(i, m) \in I_A$, and Lemmas 1–2 below to derive the later analysis of regret bounds.

$$T_{im}(t_s) \geq 90 \frac{C_u}{C_l} \ln T, \quad l_s \geq 90 C_u \ln T \quad (7)$$

$$90 C_u \ln T \geq t_1 = \mathcal{O}\left(\frac{C_u^2 N \ln T}{C_l}\right).$$

$$\Pr[|\hat{r}_{im}(t) - \bar{r}_{im}| \geq d_{im}^r(t)] \leq \frac{2}{t^2}$$

$$\Pr[|\hat{c}_{im}(t) - \bar{c}_{im}| \geq d_{im}^c(t)] \leq \frac{4}{t^2} \quad (8)$$

$$\Pr[q_{im} \leq \hat{q}_{im}(t)] \leq 1 - \frac{6}{t^2}.$$

Lemma 1. *For any $s \geq 1$ and $(i, m) \in A'_s$, the phase length $l_s \leq 2C_u(T_{im}(t_{s+1}) - T_{im}(t_s))$ and the accumulated execution times $T_{im}(t_{s+1}) \leq 4T_{im}(t_s)$.*

Lemma 2. *For any $s \geq 1$, there exists $(i, m) \in A'_s$, such that $T_{im}(t_{s+1}) \geq (1 + \frac{C_l}{C_u})T_{im}(t_s)$. As a result, for all $s \geq 1$, $t_s \geq C_l(1 + \frac{C_l}{C_u})^{\frac{s}{NM}-2}$.*

Lemma 2 and that $\ln(1+x) \geq x/2$ for any $x \in [0, 1]$ infer that $\ln t_s \geq \ln C_l + (\frac{s}{NM} - 2) \ln(1 + \frac{C_l}{C_u}) > \ln C_l + (\frac{s}{NM} - 2) \frac{C_l}{2C_u}$. And hence

$$S \leq NM \left(\frac{2C_u}{C_l} \ln T + 2 \right) + 1 = \mathcal{O}\left(\frac{C_u}{C_l} NM \ln T\right). \quad (9)$$

We let $a^* \in \arg\max_{a \in \mathcal{A}} \sum_{i,m} a_{im} q_{im}$, $A^* = \{(i, m) \in I_A \mid a_{im}^* = 1\}$. We then define the sub-optimal gaps $\Delta_a, \Delta_{im}, \underline{\Delta}, \underline{\Delta}^*$ as below:

$$\Delta_a^\alpha = \frac{1}{1+\alpha} \sum_{a_{im}^*=1} q_{im} - \sum_{a_{im}=1} q_{im} \quad \forall a \in \mathcal{A}$$

$$\Delta_{im}^\alpha = \min_{a \in \mathcal{A} \mid a_{im}=1, \Delta_a^\alpha > 0} \Delta_a \quad \forall (m, i) \in I_A \quad (10)$$

$$\underline{\Delta}^\alpha = \min_{(m,i) \in I_A} \Delta_{im}^\alpha, \quad \underline{\Delta}^* = \min_{(m,i) \in I_A \setminus A^*} \Delta_{im}^0$$

Further, we denote the variance of the processing time $c_{t,im}$ as $\sigma_{im}^2 = \mathbf{E}[(c_{t,im} - \bar{c}_{im})^2]$, and thus

$$\sigma_{im}^2 \leq \mathbf{E}[(c_{t,im} - C_l)^2] \leq (C_u - C_l) \mathbf{E}[c_{t,im} - C_l] = (C_u - C_l)(\bar{c}_{im} - C_l). \quad (11)$$

A. Optimal Regret

This heterogeneous task assignment problem is equivalent to a single-agent task-assignment problem with NM tasks and maximal capacity $\bar{L}$, with slight modification on the set of actions to avoid performing the same task redundantly.

When we can solve the linear optimization problem (6) exactly, by leveraging [2, Theorem 4.1 and 4.5], we then have the following upper and lower bounds on the optimal regret R_T^0 .

Theorem 1. *The optimal regret R_T^0 for Algorithm 1 is bounded above by*

$$\begin{aligned} R_T^0 &\leq \mathcal{O}\left(\bar{L} \sum_{\forall m,i} \frac{\tilde{C}_{im} \ln T}{\Delta_{im}^0} + \frac{C_u^2}{C_l^2} NM \bar{L} \ln T\right) \\ &\leq \mathcal{O}\left(\frac{\max_{\forall m,i} \tilde{C}_{im}}{\underline{\Delta}^*} NM \bar{L} \ln T + \frac{C_u^2}{C_l^2} NM \bar{L} \ln T\right) \\ &\leq \mathcal{O}\left(\left(\frac{1}{\underline{\Delta}^*} + C_u\right) \frac{C_u}{C_l^2} NM \bar{L} \ln T\right), \end{aligned}$$

where

$$\begin{aligned} \tilde{C}_{im} &\leq \mathcal{O}\left(\frac{1}{\bar{c}_{im}} \left(1 + \frac{1}{\bar{c}_{im}^2} \left(\sigma_{im}^2 + \frac{(C_u - C_l)^2 C_l}{C_u}\right)\right)\right) \\ &\leq \mathcal{O}\left(\frac{1}{\bar{c}_{im}} \left(1 + \frac{C_u - C_l}{\bar{c}_{im}}\right)\right) \leq \mathcal{O}\left(\frac{C_u}{\bar{c}_{im} C_l}\right). \end{aligned} \quad (12)$$

Further, for any distribution, the optimal regret is bounded as $R_T^0 = \mathcal{O}\left(\frac{1}{C_l} \sqrt{C_u NM \bar{L} T} \ln T + \frac{C_u^2}{C_l^2} NM \bar{L} \ln T\right)$. Moreover, for any N, M, T, L such that $T = \Omega(C_u)$, there exists a problem instance for which the regret of any algorithm is lower bounded by $R_T^0 = \Omega\left(\frac{1}{C_l} \min\{\sqrt{C_u NM \bar{L} T}, \bar{L} T\}\right)$

In the proposition below, we show a special case in which we can find an optimal solution to the MILP (6) by solving just a linear program (LP).

Proposition 2. *When $K_m f_{im} = L_m$ for some $K_m \in \mathbb{N}$ for all $m \in I_M$, then the linear program below always has an integer solution.*

$$\begin{aligned} &\max_a \Sigma(\hat{q}(t_s) \odot a) \\ &\sum_{i=1}^N f_{im} a_{im} \leq L_m \quad \forall m \in I_M, \\ &\sum_{m=1}^M a_{im} \leq 1 \quad \forall i \in I_V, \quad a_{im} \in [0, 1] \quad \forall (i, m) \in I_A \end{aligned} \quad (13)$$

Proof. We vectorize a as $\vec{a}$ with length NM such that $\vec{a}_{iN+m} = a_{im}$ and show the results through the property of TUM. One can verify that we can rewrite the constraints in (13) as $[E^\top \quad \mathbf{I}_N \quad -\mathbf{I}_N]^\top \vec{a} \leq [\vec{b}^\top \quad \mathbf{1}_{NM}^\top \quad \mathbf{0}_{NM}^\top]^\top$, where E is a $(M+N) \times MN$ matrix and $\vec{b}$ is a vector of length $(M+N)$:

$$E = \begin{bmatrix} \mathbf{1}_N^\top & \mathbf{0}_N^\top & \cdots & \mathbf{0}_N^\top \\ \mathbf{0}_N^\top & \mathbf{1}_N^\top & \cdots & \mathbf{0}_N^\top \\ \vdots & \ddots & \ddots & \vdots \\ \mathbf{0}_N^\top & \mathbf{0}_N^\top & \cdots & \mathbf{1}_N^\top \\ \mathbf{I}_N & \mathbf{I}_N & \cdots & \mathbf{I}_N \end{bmatrix} \text{ and } \vec{b} = \begin{bmatrix} K_1 \\ K_2 \\ \vdots \\ K_M \\ \mathbf{1}_N \end{bmatrix}.$$

By Hoffman's sufficient conditions, E is a TUM, and by the property that concatenation of TUM with identity matrix and changing sign preserve TUM, we know that

$[E^\top \quad \mathbf{I}_N \quad -\mathbf{I}_N]^\top$ is a TUM. The proposition then follows directly from the property of TUM. $\square$

Proposition 2 shows that we are able to obtain an integer optimal solution of the LP, and in fact, if we solve (13) through the simplex algorithm, it will return an integer optimal solution, implying that we are able to solve the MILP problem (6) efficiently by solving a LP.

Remark 2. *In most cases, we can solve the MILP (6) and select the best task assignment problem only if the problem size is small (N, M are small values), given its NP-hardness. However, Proposition 2 provides us with an efficient, exact, and scalable computation method for it under some special scenarios.*

B. Sub-optimal Regret

In this section, we show the upper bound of the α -regret for Algorithm 1. Through Theorem 2 below, we learn how good the approximated algorithm can perform.

Theorem 2. *Given that Algorithm 1 is an α approximating algorithm, the sub-optimal regret R_T^α where $\alpha \geq 1$ for Algorithm 1 is bounded above by*

$$\begin{aligned} R_T &\leq \mathcal{O}\left(\bar{L} \sum_{\forall m,i} \frac{\tilde{C}_{im} \ln T}{\Delta_{im}^\alpha} + \frac{C_u^2}{C_l^2} NM \bar{L} \ln T\right) \\ &\leq \mathcal{O}\left(\frac{\max_{\forall m,i} \tilde{C}_{im}}{\underline{\Delta}^\alpha} NM \bar{L} \ln T + \frac{C_u^2}{C_l^2} NM \bar{L} \ln T\right) \\ &\leq \mathcal{O}\left(\left(\frac{1}{\underline{\Delta}^\alpha} + C_u\right) \frac{C_u}{C_l^2} NM \bar{L} \ln T\right), \end{aligned} \quad (14)$$

where $\tilde{C}_{im}$ is defined in (12).

Further, for any distribution, the sub-optimal regret is bounded as $R_T = \mathcal{O}\left(\frac{1}{C_l} \sqrt{C_u NM \bar{L} T} \ln T + \frac{C_u^2}{C_l^2} NM \bar{L} \ln T\right)$.

Proof Sketch: We utilize the proving method in [2] with modification. Given (4), we have

$$\begin{aligned} R_T &\leq \mathbb{E} \left[\sum_{t=1}^T \Sigma \left(\frac{1}{\alpha+1} q \odot a^* - r_t \odot a_t \right) \right] + \frac{C_u \bar{L}}{(\alpha+1)C_l} \\ &\leq \frac{\bar{L}}{(\alpha+1)C_l} \mathbb{E}[t_1] + R_T^{(1)} + R_T^{(2)} + \frac{C_u \bar{L}}{(\alpha+1)C_l} \end{aligned} \quad (15)$$

where

$$\begin{aligned} R_T^{(1)} &= \mathbb{E} \left[\sum_{s=1}^S \sum_{t=t_s}^{t_{s+1}-1} \Sigma \left(\frac{1}{\alpha+1} q \odot a^* - q \odot a'_s \right) \right] \\ R_T^{(2)} &= \mathbb{E} \left[\sum_{s=1}^S \sum_{t=t_s}^{t_{s+1}-1} \Sigma \left(q \odot a'_s - r_t \odot a_t \right) \right] \end{aligned}$$

and we define S as the phase that includes round T , $t_{S+1} = T+1$ for convenience. We then show that

$$R_T^{(1)} = \mathcal{O}\left(\bar{L} \sum_{\forall (i,m) \in I_A} \frac{\tilde{C}_{im} \ln T}{\Delta_{im}^\alpha} \ln T + \frac{C_u^2}{C_l^2} NM \bar{L}\right) \quad (16)$$

$$R_T^{(2)} = \mathcal{O}\left(\frac{C_u^2}{C_l^2} N M \bar{L} \ln T\right). \quad (17)$$

(15)–(17) together show the gap-dependent bound in (14). Notice that (16) utilizes the fact that $\Sigma(\hat{q}(t_s) \odot a'_s) \geq \frac{1}{1+\alpha} \Sigma(\hat{q}(t_s) \odot a^*)$ in the proof. Lastly, We obtain the gap-free regret bound by modifying the analysis of $R_T^{(1)}$. $\square$

V. NUMERICAL STUDY

We demonstrate the proposed bandit heterogeneous task-assigning algorithm on a case study with 3 teams of different sizes. We perform the simulation using Matlab, where we use Gurobi with Yalmip toolbox to compute the exact solution for (6) at the beginning of each phase s , and use the algorithm proposed in [20]—described below in Algorithm 2—as Algorithm A in line (4) of Algorithm 1. Algorithm 2 will guarantee to find a 2-approximate solution to (6). Therefore, Algorithm 1 is an exact algorithm when directly computing (6) and a 2-approximate algorithm when using 2. We therefore simplify this 2-approximate algorithm as the approximate algorithm in the case study².

Algorithm 2 Approximate Task Assigning at Round t_s

Input: $\hat{q}_{im}(t_s)$ all $(i, m) \in I_A$, L_m for all $m \in I_M$
for $s = 1, 2, \dots$ **do**
 Initialization: let $Z \in \mathbb{N}^N$, and set $Z_i = -1$ for all $i \in I_V$. Let $P \in \mathbb{R}^{N \times M}$.
 for $m = 1, \dots, M$ **do**
 For $i \in I_V$, set $P_{im} = \hat{q}_{im}(t_s)$ if $Z[i] = -1$, and set $P_{im} = \hat{q}_{im}(t_s) - \hat{q}_{iZ_i}(t_s)$ otherwise.
 Find $s^* \in \{\text{argmax}_s \sum_i P_{im} s_i \mid \sum_{i=1}^N s_i \leq L_m, s_i \in \{0, 1\} \ \forall i \in I_V\}$
 For all i such that $s_i^* = 1$, set $Z_i = m$.
 end for
end for
For all $(i, m) \in I_A$, $a'_{s,iZ_i} = 1$ if $Z_i \neq -1$

The small team has $N = 4$ tasks, $M = 2$ team members, $L = \{2, 1\}$, $\bar{r}_{im} = 0.5$ for all i, m except that $r_{12} = 0.7$, $\bar{c}_{im} = 1.5$ for $i = 1, 2, m = 1, 2$, and $\bar{c}_{im} = 5$ for $i = 3, 4, m = 1, 2$, $f_{im} = 1$ for all i, m except that $f_{21} = 1.4$ and $f_{31} = 1.8$. The medium team has $N = 20$ tasks, $M = 5$ team members, and $L = \{2, 5, 4, 5, 6\}$. The large team has $N = 20$ tasks, $M = 10$ team members, and $L_m = 4$ for all m . We provide mean rewards and completion times and resource occupation for the medium and the large teams with code given the size of the problem. We set $T = 10000$ for all three teams.

The approximate algorithm can assign the tasks properly to the teams of all 3 sizes at each round in a timely manner, while the exact algorithm can only work for the smallest team. Fig. 2b demonstrates the cumulated sub-optimal regret R_T^1 of the 3 teams with different sizes for the approximate

algorithm. For the smallest team, we also compute the cumulative optimal regret R_T^0 for the exact algorithm in comparison to R_T^1 for the approximate algorithm in Fig. 2a.

The cumulative optimal regret R_T^0 for the exact algorithm always scales logarithmically with T (the red curve in Fig. 2a), while the sub-optimal regret R_T^1 for the approximate algorithm goes negative and scales linearly with time (the blue curve in Fig. 2a, all curves in Fig. 2b and Fig. 2c). The reason for these negative, linear sub-optimal regrets is that the approximate algorithm can outperform the benchmark $\frac{1}{2}OPT$, where OPT is the optimal accumulated reward that the team may obtain.

Fig. 2c, we are able to observe that the overall learning rate decreases when the lower and upper bounds become loose. Both the green and the blue curves represent the cumulative sub-optimal regret for the medium team. While keeping all other parameters the same, we change the completion bounds from $[C_l, C_u] = [3, 10]$ (green) to $[C_l, C_u] = [1, 20]$ (blue), and the demonstrated degradation in performance is potentially caused by the large UCBs on q .

Finally, in Fig. 2d, we show that the large team has the sub-optimal regret R_T^1 for the approximate algorithm also scaling logarithmically with T , as the approximate algorithm can perform as good as the exact algorithm, depending on the detailed parameters.

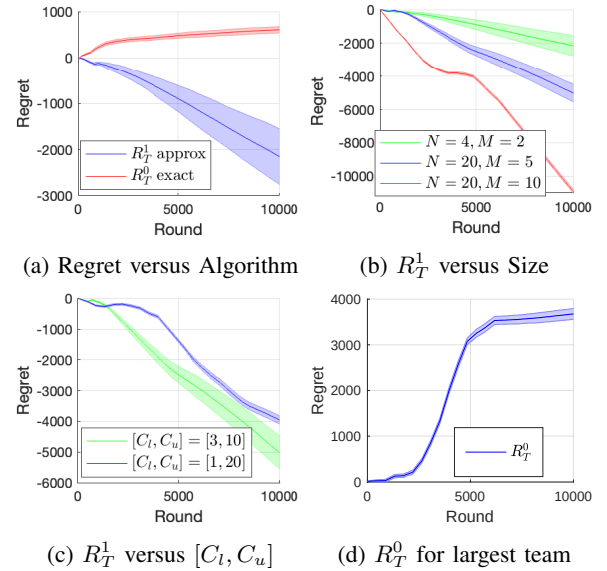


Fig. 2: Cumulative regrets scale as logarithmic in Fig. 2a (exact method curve in red) and Fig. 2d, and negative linear regret in Fig. 2a, (exact method curve in red), Fig. 2c, and Fig. 2d.

VI. CONCLUSION

We introduced the heterogeneous bandit task assignment problem in a multi-agent setting and studied an efficient task assignment algorithm for large-scale problems where computing the exact optimal solution at each round is intractable. We theoretically analyzed the optimal and sub-optimal regret

²All our code and data used in the case study can be found in <https://github.com/QinshuangCoolWei/Heterogeneous-Task-Assignment-with-Uncertain-Execution-Times-and-Preferences.git>.

bounds for both the exact and the approximate algorithms, and demonstrated the effectiveness of the algorithm via case studies with teams of varying sizes.

The proposed task assignment algorithm adapts to heterogeneous teammate preferences and abilities by modeling them as fixed distributions. However, in practice, these attributes can evolve over time—individual proficiency and interest in tasks may fluctuate based on time, rewards, etc. Therefore, a future direction is to incorporate these evolving preferences and abilities into the task assignment process, enabling more adaptive and satisfying at all times.

REFERENCES

- [1] S. Basu, R. Sen, S. Sanghavi, and S. Shakkottai, “Blocking bandits,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [2] S. Ito, D. Hatano, H. Sumita, K. Takemura, T. Fukunaga, N. Kakimura, and K.-i. Kawarabayashi, “Bandit task assignment with unknown processing time,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [3] H. Zhang, Y. Ma, and M. Sugiyama, “Bandit-based task assignment for heterogeneous crowdsourcing,” *Neural computation*, vol. 27, no. 11, pp. 2447–2475, 2015.
- [4] U. ul Hassan and E. Curry, “Efficient task assignment for spatial crowdsourcing: A combinatorial fractional optimization approach with semi-bandit learning,” *Expert Systems with Applications*, vol. 58, pp. 36–56, 2016.
- [5] P. García, P. Caamaño, R. J. Duro, and F. Bellas, “Scalable task assignment for heterogeneous multi-robot teams,” *International journal of advanced robotic systems*, vol. 10, no. 2, p. 105, 2013.
- [6] B. Schwartz, A. Ward, J. Monterosso, S. Lyubomirsky, K. White, and D. R. Lehman, “Maximizing versus satisficing: happiness is a matter of choice,” *Journal of personality and social psychology*, vol. 83, no. 5, p. 1178, 2002.
- [7] X. Wang, J. Ye, and J. C. Lui, “Decentralized task offloading in edge computing: A multi-user multi-armed bandit approach,” in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1199–1208.
- [8] R. Marjeh, A. Gokhale, F. Bullo, and L. Griffiths, “Task allocation in teams as a multi-armed bandit,” *ACM Collective Intelligence*, 2024.
- [9] U. U. Hassan and E. Curry, “A multi-armed bandit approach to online spatial task assignment,” in *2014 IEEE 11th Intl Conf on Ubiquitous Intelligence and Computing and 2014 IEEE 11th Intl Conf on Autonomic and Trusted Computing and 2014 IEEE 14th Intl Conf on Scalable Computing and Communications and Its Associated Workshops*. IEEE, 2014, pp. 212–219.
- [10] H. Zhang and M. Sugiyama, “Task selection for bandit-based task assignment in heterogeneous crowdsourcing,” in *2015 Conference on Technologies and Applications of Artificial Intelligence (TAAI)*. IEEE, 2015, pp. 164–171.
- [11] A. Rangi and M. Franceschetti, “Multi-armed bandit algorithms for crowdsourcing systems with online estimation of workers’ ability,” in *AAMAS*, 2018, pp. 1345–1352.
- [12] L. Tran-Thanh, S. Stein, A. Rogers, and N. R. Jennings, “Efficient crowdsourcing of unknown experts using bounded multi-armed bandits,” *Artificial Intelligence*, vol. 214, pp. 89–111, 2014.
- [13] Y. Xiao, Y. Yamashita, H. Ito, M. Matsubara, and A. Morishima, “Multi-armed bandit approach to qualification task assignment across multi crowdsourcing platforms,” in *2022 IEEE International Conference on Big Data (Big Data)*. IEEE, 2022, pp. 4040–4048.
- [14] Y. Zhao, J. Xia, G. Liu, H. Su, D. Lian, S. Shang, and K. Zheng, “Preference-aware task assignment in spatial crowdsourcing,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 2629–2636.
- [15] Y. Zhao, K. Zheng, H. Yin, G. Liu, J. Fang, and X. Zhou, “Preference-aware task assignment in spatial crowdsourcing: from individuals to groups,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 7, pp. 3461–3477, 2020.
- [16] J. Fruitet, A. Carpentier, R. Munos, and M. Clerc, “Automatic motor task selection via a bandit algorithm for a brain-controlled button,” *Journal of neural engineering*, vol. 10, no. 1, p. 016012, 2013.

- [17] N. Cesa-Bianchi and G. Lugosi, “Combinatorial bandits,” *Journal of Computer and System Sciences*, vol. 78, no. 5, pp. 1404–1422, 2012.
- [18] B. Kveton, Z. Wen, A. Ashkan, and C. Szepesvari, “Tight regret bounds for stochastic combinatorial semi-bandits,” in *Artificial Intelligence and Statistics*. PMLR, 2015, pp. 535–543.
- [19] Z. Dong, S. Das, P. Fowler, and C.-J. Ho, “Efficient nonmyopic online allocation of scarce reusable resources,” in *AAMAS Conference proceedings*, 2021.
- [20] R. Cohen, L. Katzir, and D. Raz, “An efficient approximation for the generalized assignment problem,” *Information Processing Letters*, vol. 100, no. 4, pp. 162–166, 2006.

APPENDIX I PROOF FOR PROPOSITION 1

Proof. Our multi-member task assignment setup with N tasks and M members with at most $\bar{L}$ simultaneous tasks is equivalent to that for a single agent task assignment with NM tasks and $\bar{L}$ maximal simultaneous tasks. As a result, we can directly apply the proof for [2, Proposition 2.2] to Proposition 1 here. $\square$

APPENDIX II PROOF FOR LEMMA 1

Proof. Given (7), we then have $\min_{(i,m) \in A'_s} T_{im}(t_s) \geq 2\frac{C_u}{C_l}$, and thus $l_s = C_l \min_{(i,m) \in I_A} T_{im}(t_s) + 2C_u \geq 4C_u$. Therefore, for all $(i, m) \in A'_s$, the completion number is bounded from below $T_{im}(t_{s+1}) - T_{im}(t_s) \geq (l_s - 2C_u)/C_u$. Notice that $l_s - 2C_u - l_s/2 = l_s/2 - 2C_u \geq 4C_u/2 - 2C_u \geq 0$. Hence $(l_s - 2C_u)/C_u \geq l_s/(2C_u)$, and we can conclude that $l_s \leq 2C_u(T_{im}(t_{s+1}) - T_{im}(t_s))$.

Similarly, the completion number is bounded from above $T_{im}(t_{s+1}) - T_{im}(t_s) \leq (l_s + 1)/C_l = (C_l \min_{(i,m) \in I_A} T_{im}(t_s) + 2C_u + 1)/C_l = \min_{(i,m) \in I_A} T_{im}(t_s) + 2C_u/C_l + 1/C_l \leq 3T_{im}(t_s)$, where the last inequality follows from that $2C_u > 1$ and that $\min_{(i,m) \in A'_s} T_{im}(t_s) \geq 2\frac{C_u}{C_l}$. $\square$

APPENDIX III PROOF FOR LEMMA 2

Proof. We let $(i^*, m^*) \in \operatorname{argmin}_{(i,m) \in A'_s} T_{im}(t_s)$. Then $l_s = T_{i^*, m^*}(t_s) + 2C_u$. The member m^* can complete the task i^* for at least $(l_s - 2C_u)/C_u = (C_l/C_u)T_{i^*, m^*}(t_s)$ times in phase s , and hence $T_{i^*, m^*}(t_{s+1}) \geq (1 + C_l/C_u)T_{i^*, m^*}(t_s)$. If $(i, m) \in A'_s$, for some $s' < s$, then (7) indicates that $T_{im}(t_s) \geq \frac{C_u}{C_l}$. Then if for some $K \in \mathbb{N}$, we have $s \geq KNM$, then there exists $(i, m) \in I_A$ such that $T_{im}(t_s) \geq (C_u/C_l)(1 + C_u/C_l)^{K-1}$ (which can be easily shown by contradiction). Since $t_s \geq C_l T_{im}(t_s)$, where right hand side is the shortest time for completion, for all $(i, m) \in I_A$, we then have $t_s \geq C_l(\frac{C_u}{C_l})(1 + \frac{C_u}{C_l})^{\lfloor \frac{s}{NM} \rfloor - 1} \geq C_l(1 + \frac{C_u}{C_l})^{\frac{s}{NM} - 2}$. $\square$

APPENDIX IV PROOF FOR THEOREM 2

In this section we assume that Algorithm A in line 4 of Algorithm 1 yields a $(1 + \alpha)$ -approximate solution to (6), and that Algorithm 1 is an $(1 + \alpha)$ -approximate algorithm.

We first denote the optimal UCB task assignment at round t_s as $\hat{a}_{s,im}^*$, i.e.,

$$\hat{a}_{s,im} \in \operatorname{argmax}_{a \in \mathcal{A}} \Sigma(\hat{q}(t_s) \odot a) \quad (18)$$

We further define the index set for the optimal and chosen assignments

$$\begin{aligned} A'_s &= \{(i, m) \in I_{\mathcal{A}} \mid a'_{s,im} = 1\}, \\ \hat{A}_s &= \{(i, m) \in I_{\mathcal{A}} \mid \hat{a}_{s,im} = 1\}. \end{aligned} \quad (19)$$

Notice that, based on our assumption that Algorithm A yields a $(1 + \alpha)$ -approximate solution to (6), we have

$$\frac{1}{1 + \alpha} \sum_{\hat{a}_{s,im}^* = 1} \hat{q}_{im}(t_s) \leq \sum_{a'_{s,im} = 1} \hat{q}_{im}(t_s). \quad (20)$$

For simplicity of expression, we define the quantity

$$d_{im}(t) = \sqrt{\frac{\tilde{C}_{im}}{\bar{c}_{im}} \frac{\ln t}{T_{im}(t)}} \quad (21)$$

and a designated set of phases

$$\mathcal{F}_s = \{s \geq 1 \mid \Delta_{a'_s}^\alpha \leq \sum_{(i,m) \in A'_s} d_{im}(t_s), \Delta_{a'_s}^\alpha > 0\}, \quad (22)$$

and let $\bar{\mathcal{F}}_s$ be its complement set. Finally, we define the reward our algorithm can obtain from the phases in (22) as $\hat{R}_T$

$$\hat{R}_T = \sum_{s=1}^S l_s \Delta_{a'_s}^\alpha \mathbb{1}[\mathcal{F}_s]. \quad (23)$$

Lemma 3. *For all phases $s \geq 1$, we have $\Delta_{a'_s}^\alpha \leq \sum_{(i,m) \in A'_s} d_{im}(t_s)$ with a probability at least $1 - \frac{\bar{L}}{t_s^2}$. We then have $R_T^{(1)} \leq \mathbb{E}[\hat{R}_T] + \mathcal{O}(\frac{MN\bar{L}}{C_l t_1})$*

Proof. We first show $\Delta_{a'_s}^\alpha \leq \sum_{(i,m) \in A'_s} d_{im}(t_s)$ with a probability $1 - \frac{6MN}{t_s^2}$ for all $s \geq 1$.

By revising the proof in [2, Lemma 4.3] task $i \in I_{\mathcal{V}}$ to agent-task pair $(i, m) \in I_{\mathcal{A}}$, we first have (24)–(26) below.

$$\begin{aligned} \Pr[|\hat{r}_{im}(t) - \bar{r}_{im}| \geq d_{im}^r(t)] &\leq \frac{2}{t^2} \\ \Pr[|\hat{c}_{im}(t) - \bar{c}_{im}| \geq d_{im}^c(t)] &\leq \frac{4}{t^2} \end{aligned} \quad (24)$$

$$\Pr[q_{im} \leq \hat{q}_{im}(t)] \leq 1 - \frac{6}{t^2},$$

$$\hat{q}_{im}(t_s) \leq q_{im}(t_s) + d_{im}(t_s), \quad (25)$$

and that

$$\frac{l_s}{t_s^2} \leq \frac{5}{t_1}. \quad (26)$$

We first note that

$$\Delta_{a'_s}^\alpha \leq \frac{1}{1 + \alpha} \sum_{a'_{s,im} = 1} q_{im} \leq \frac{\bar{L}}{(1 + \alpha)C_l} \quad (27)$$

as $q_{im} \leq 1$, and there exists at most $\bar{L}$ pairs of (i, m) such that $a'_{s,im} = 1$.

With probability at least $1 - \frac{6NM}{t^2}$, we then have

$$\begin{aligned} \Delta_{a'_s}^\alpha &= \frac{1}{1 + \alpha} \sum_{a'_{s,im} = 1} q_{im} - \sum_{a'_{s,im} = 1} q_{im} \\ &\leq \frac{1}{1 + \alpha} \sum_{a'_{s,im} = 1} \hat{q}_{im}(t_s) - \sum_{a'_{s,im} = 1} q_{im} \end{aligned} \quad (28)$$

$$\leq \frac{1}{1 + \alpha} \sum_{\hat{a}_{s,im} = 1} \hat{q}_{im}(t_s) - \sum_{a'_{s,im} = 1} q_{im} \quad (29)$$

$$\begin{aligned} &= \frac{1}{1 + \alpha} \sum_{\hat{a}_{s,im} = 1} \hat{q}_{im}(t_s) - \sum_{a'_{s,im} = 1} \hat{q}_{im}(t_s) \\ &\quad + \sum_{a'_{s,im} = 1} (\hat{q}_{im}(t_s) - q_{im}(t_s)) \\ &\leq \sum_{a'_{s,im} = 1} \hat{q}_{im}(t_s) - \sum_{a'_{s,im} = 1} \hat{q}_{im}(t_s) + \sum_{a'_{s,im} = 1} d_{im}(t_s) \end{aligned} \quad (30)$$

where (28) is true with probability $1 - \frac{6NM}{t^2}$ based on (24). (29) holds based on the optimum definition of $\hat{a}_{s,im}^* = 1$ in (18), and (30) follows immediately from (20).

Next, we rewrite and bound $R_T^{(1)}$ in the rest of the proof.

$$\begin{aligned} R_T^{(1)} &= \mathbb{E} \left[\sum_{s=1}^S \sum_{t=t_s}^{t_{s+1}-1} \Sigma \left(\frac{1}{\alpha + 1} q \odot a^* - q \odot a'_s \right) \right] \\ &= \mathbb{E} \left[\sum_{s=1}^S l_s \Delta_{a'_s}^\alpha \right] \\ &= \mathbb{E} \left[\sum_{s=1}^S l_s \Delta_{a'_s}^\alpha \mathbb{1}[\mathcal{F}_s] + \sum_{s=1}^S l_s \Delta_{a'_s}^\alpha \mathbb{1}[\bar{\mathcal{F}}_s] \right] \\ &= \mathbb{E}[\hat{R}_T] + \mathbb{E} \left[\sum_{s=1}^S l_s \Delta_{a'_s}^\alpha \mathbb{1}[\bar{\mathcal{F}}_s] \right], \end{aligned} \quad (31)$$

where

$$\begin{aligned} &\mathbb{E} \left[\sum_{s=1}^S l_s \Delta_{a'_s}^\alpha \mathbb{1}[\bar{\mathcal{F}}_s] \right] \\ &= \mathbb{E} \left[\sum_{s=1}^S l_s \Delta_{a'_s}^\alpha \mathbb{1}[\Delta_{a'_s}^\alpha > \sum_{(i,m) \in A'_s} d_{im}(t_s)] \right] \\ &\quad + \mathbb{E} \left[\sum_{s=1}^S l_s \Delta_{a'_s}^\alpha \mathbb{1}[\Delta_{a'_s}^\alpha < 0] \right] \\ &\leq \mathbb{E} \left[\sum_{s=1}^S l_s \frac{\bar{L}}{(1 + \alpha)C_l} \frac{6MN}{t_s^2} \right] = \frac{6MN\bar{L}}{(1 + \alpha)C_l} \mathbb{E} \left[\frac{l_s}{t_s^2} \right] \end{aligned} \quad (32)$$

$$\leq \frac{30MN\bar{L}}{(1 + \alpha)C_l t_1}, \quad (33)$$

where (32) follows from that (30) holds with probability at least $1 - \frac{6NM}{t^2}$ and (27), and (33) follows from (26).

Then (31) and (33) together yields the bound on $R_T^{(1)}$ in Lemma 3. $\square$

Lemma 4. *For all $s \leq 1$ and $(i, m) \in A'_s$, we have*

$$\mathbb{E}[T_{im}(t_{s+1}) - T_{im}(t_s) \mid l_s] \geq \frac{l_s - 2C_u}{\bar{c}_i}. \quad (34)$$

Therefore, we have

$$l_s \leq 2\bar{c}_i \mathbb{E}[T_{im}(t_{s+1}) - T_{im}(t_s) \mid l_s, T_{im}(t_s)]. \quad (35)$$

$$\textbf{Lemma 5. } \mathbb{E}[\hat{R}_T] = \mathcal{O}\left(\bar{L} \sum_{(i,m) \in A'_s} \frac{\tilde{C}_{im} \ln T}{\Delta_{im}}\right)$$

Proof.

□

$$\textbf{Lemma 6. } R_T^{(2)} = \mathbb{E}\left[\sum_{s=1}^S \sum_{t=t_s}^{t_{s+1}-1} \Sigma(q \odot a'_s - r_t \odot a_t)\right] = \mathcal{O}\left(\frac{C_u^2}{C_l^2} NM\bar{L} \ln T\right)$$

Proof. We first show that $\mathbb{E}\left[\sum_{t=t_s}^{t_{s+1}-1} \Sigma(q \odot a'_s - r_t \odot a_t)\right] \leq 3\bar{L} \frac{C_u}{C_l}$ given any a'_s, t_s , and l_s . □

Proof for Theorem 2

Proof.

$$R_T^{(1)} = \mathbb{E}\left[\sum_{s=1}^S \sum_{t=t_s}^{t_{s+1}-1} \Sigma\left(\frac{1}{\alpha+1} q \odot a^* - q \odot a'_s\right)\right]$$

$$R_T^{(1)} = \mathcal{O}\left(\bar{L} \sum_{\forall (i,m) \in I_A} \frac{\tilde{C}_{im} \ln T}{\Delta_{im}^\alpha} \ln T + \frac{C_u^2}{C_l^2} NM\bar{L}\right) \quad (36)$$

□